How Indexes Accelerate Queries

Speaker manuscript

SKILL Application Test

Slide 1 How Indexes Accelerate Queries

We will use a concrete query to understand what an index changes inside a database. The query asks for the 20 most recent orders belonging to a particular tenant. Its result stays fixed while we change the available index. This lets us connect a data structure to a query plan and then to the work performed during execution.

The story begins with a full scan. We then add a tenant key, combine that key with the requested order, and extend it to supply the output values. Along the way, we will look at column order, partial indexes, expression predicates and maintenance costs. The goal is to choose an index by understanding the work that the query actually needs.

Slide 2 Start with one query and a fixed result

Read the SQL from the filter outward. We select orders for tenant 42, order them by descending creation sequence, and return the first 20 values of created_seq and amount_cents. The creation sequence is an increasing integer that represents creation order. The amount is stored as integer cents. Each row also has a 64-byte payload field.

The experiment contains 200,000 synthetic orders spread across 100 tenants. Tenant 42 has 1,980 rows. We measure SQLite virtual-machine work using a progress callback at an instruction interval of 1, and separately time complete result retrieval after warm-up. Every comparison checks the returned rows and their order. Keeping that check in place makes a change in execution work interpretable as an access-path change for the same requested answer.

Slide 3 An index stores an additional order over the data

These small illustrative tables show the idea before we examine a larger tree. The table on the left is arranged by row ID. The index on the right is arranged first by tenant and then by row ID. The same logical record can therefore be located through a different ordering.

For tenant 42, the corresponding index entries are adjacent. After finding those entries, their row IDs identify the table records that contain additional values. This gives the index 2 roles: its leading key helps narrow the search, and its row identifiers connect the search result to the remaining table data. The exact order of key fields becomes important when a query includes several predicates or requests a particular output order.

Slide 4 A B tree routes the search through a few pages

The diagram represents storage pages as nodes. The upper page contains separator keys 30 and 60. A search for 42 follows the middle range because 42 lies between those separators. The selected lower page then contains the matching entry.

A real database page can hold many keys and child links. That branching capacity is called fan-out. It lets a tree represent a large number of entries while keeping the route from the root to a

target region short. The diagram uses a small number of illustrative keys so that the route is easy to trace. The next page will connect this hierarchical idea to the actual page inventory measured for an index in our SQLite experiment.

Slide 5 High fan out keeps the tree shallow

The measured index has 6 internal pages and 846 leaf pages. Its longest root-to-leaf path contains 2 edges, so the tree has 3 levels. The internal pages contain 845 entries and the leaves contain 199,155 entries, totaling the 200,000 indexed records. This page inventory comes from SQLite's dbstat view.

The storage figures distinguish the small internal routing structure from the much larger leaf region. The internal pages occupy 24 KiB, while the leaves occupy about 3.305 MiB. A selective lookup follows the tree toward a relevant key region. A full scan instead traverses the chosen table or index across its entries. That distinction prepares us to read SCAN and SEARCH in the execution plans that follow.

Slide 6 Without an index filter and sort the table

The baseline has no secondary index for the tenant predicate. Follow the sequence in the diagram: inspect the table rows, identify the 1,980 rows belonging to tenant 42, and determine which 20 have the largest creation sequences. The result is small, but finding the eligible recent rows requires substantial work.

EXPLAIN QUERY PLAN reports a scan of orders and a temporary B-tree for ORDER BY. These descriptions expose the broad access path and the separate ordering work. In the measured run, the query executes 617,851 virtual-machine instructions to retrieve its 20 rows. We will keep this baseline result and compare it with each index design. The diagram and plan provide the mechanism behind the number.

Slide 7 A tenant index removes most filtering work

The first added index contains tenant_id. It makes all entries for the selected tenant accessible through an equality search. The plan now reports SEARCH orders USING INDEX idx with a tenant equality constraint. The question mark in the plan describes the equality lookup value.

The query still requests descending creation sequence, while this index is ordered by tenant and row ID. It therefore needs table records for the remaining values and separate ordering work. The temporary B-tree for ORDER BY remains in the plan. Even so, the measured instruction count falls to 23,792. We have removed much of the filtering work by narrowing the search to a tenant, while the requested order gives us the next design opportunity.

Slide 8 A composite index groups and orders together

A composite key uses more than one field. The index shown here starts with tenant_id and then uses created_seq. Read the illustrative rows lexicographically: tenant is compared first; sequence determines order among entries whose tenant values are equal.

This places tenant 42 in one contiguous region. Within that region, creation sequences are ordered from smaller to larger values. The equality constraint fixes the leading key, so the following sequence field can supply the order requested by the query. This is the important relationship

between predicates and ordering: the meaning of a later key field depends on which earlier key fields are fixed by the query.

Slide 9 An ordered index can stop after the first 20 matches

Once the tenant range is ordered by creation sequence, the database can seek to its most recent end and walk entries in descending order. It returns rows as it goes and stops after 20 matches. The requested sort order now follows directly from the index traversal.

The plan reports the tenant search without a separate temporary B-tree for ordering. Measured work falls to 150 instructions. Compare this with the earlier plan that gathered tenant rows and then determined their order. The combination of the equality predicate, the key order and LIMIT allows early termination. This explains the large reduction in work even though both plans use an index and return exactly the same values.

Slide 10 A covering index supplies every requested value

The next index adds amount_cents after tenant_id and created_seq. Each field now has a role in the query: tenant supplies the equality search, sequence supplies order and an output value, and amount supplies the other output value.

The plan reports USING COVERING INDEX because the required values can be obtained directly from the index. Table lookups for the selected amounts are removed. The measured count drops from 150 to 129 instructions. This is a refinement of an already selective and ordered path. The much larger earlier reductions came from avoiding a full scan and a separate sort; coverage removes the remaining table-access work for this query.

Slide 11 Column order determines the searchable prefix

This page switches to a range aggregate. The SQL sums amount_cents for tenant 42 over creation sequences 80,000 through 119,999. Both compared indexes contain tenant and sequence, but their order is reversed. INDEXED BY selects the intended path so that the experiment isolates that ordering decision.

With tenant first, the search enters one tenant's key range and narrows it by the sequence bounds. With sequence first, it enters the sequence range across tenants and then applies the tenant condition. The results agree, while the instruction counts are 1,991 and 200,808. The diagram-free table makes the comparison precise: a list of indexed columns is only part of the design; their order determines the constrained region that a traversal can use.

Slide 12 Coverage depends on the requested output columns

Keep the index containing tenant, sequence and amount, and compare 2 output lists. The first asks for sequence and amount, both present in the index. The second also asks for payload, which is stored in the table record.

That extra output changes the observed plan from USING COVERING INDEX to USING INDEX. The query still benefits from the tenant search and sequence order, but now it follows row IDs to fetch payload. The measured instruction count is 170 for this version. Coverage should therefore be evaluated against the actual query, including the values it needs. A change to the

requested output can change the access work even when the WHERE and ORDER BY clauses remain the same.

Slide 13 A partial index represents a useful subset

A partial index contains rows that satisfy a specified predicate. Here the index contains tenant, sequence and amount for rows whose status is open. The workload has 20,045 open orders, roughly one tenth of the complete table.

The query in this comparison explicitly includes status equal to open, so the partial index contains every row that can qualify. We compare it with a full status-first index containing status, tenant, sequence and amount. The partial index occupies about 0.336 MiB, while the full index occupies about 4.641 MiB. Both return the same active-order result. This makes the index predicate a useful part of the physical design when the application repeatedly queries a defined subset.

Slide 14 An expression can change the usable search constraint

The 2 predicates on this page return the same count in this integer dataset. The direct equality tenant_id equals 42 uses a constrained search. Wrapping the column in tenant_id plus 0 leads to a scan of the covering index in the observed plan.

The measured counts are 5,953 instructions for the direct equality and 801,994 for the expression predicate. Both return a count of 1,980 matching rows. This comparison shows why query form matters alongside index structure. When designing a query, keep the intended constraint directly searchable, or build an index for the expression that the application genuinely needs. The useful check is the resulting plan and measured work for the actual expression.

Slide 15 Read the query plan as a physical route

We can now interpret the main plan terms through examples we have already seen. SCAN describes traversal through a selected table or index. SEARCH identifies a constrained lookup, such as the tenant equality. USING COVERING INDEX tells us that the index supplies the values required by the query. USE TEMP B-TREE FOR ORDER BY identifies separate ordering work.

Read these details together rather than treating an index name as the complete result. A scan can itself traverse an index, as the expression example showed. A search can still require table lookups and sorting, as the tenant-only example showed. Pair the plan with the returned rows and the work measurements. The result confirms semantics, the plan explains the route, and the measurements quantify what that route did.

Slide 16 The same result can require radically different work

This chart collects the 4 versions of the original ordered, limited query. The horizontal axis is logarithmic, so each major step represents a factor of 10. The bars are also labeled with their measured instruction counts, which makes the smaller values directly readable.

The full scan executes 617,851 instructions. A tenant index reduces that to 23,792. Matching tenant and sequence order reduces it to 150. Adding the selected amount column gives a covering path with 129 instructions. All 4 versions return identical rows in identical order. The sequence of changes explains the outcome: first narrow the region, then obtain the order directly, then supply the remaining output values inside the index.

Slide 17 Latency and storage reveal different costs

The timing experiment uses SQLite 3.53.1 with an in-memory database containing 200,000 rows. Each plotted time is the median of 11 executions after 3 warm-up runs. The left axis is logarithmic, which allows the scan and the much shorter indexed paths to share one chart.

The right panel shows additional allocated database storage for the index designs. Another key order requires additional pages. Adding output fields to make an index covering also increases its entry width. Read the 2 panels as different dimensions of the design: one describes the observed time to retrieve the query result in this environment, and the other describes the storage needed for the added structures. The deterministic instruction counts from the previous page help connect timing to execution work.

Slide 18 Index maintenance and broad reads also consume work

An index is maintained when data changes. The left panel inserts the same 5,000 rows in one transaction for each index configuration and counts virtual-machine instructions. The transaction is rolled back after measurement so that the comparisons start from the same underlying data.

The right panel changes the fraction of rows selected by a sequence-range query and fetches payload from the table. Each access path is selected explicitly for the comparison. A small range lets the index narrow the work effectively. As the requested fraction grows, more entries and records must be visited. At 100 percent, both paths traverse the complete dataset. These measurements connect index design to the read and write operations that make up the workload.

Slide 19 Build indexes through a repeatable query experiment

Begin by writing down the exact query contract: its predicates, ordering, selected values and row limit. Capture the baseline result and plan. Then change the key design to match the operations that the query requires. This keeps the index proposal tied to a specific access path.

Measure the result on several dimensions. Check result equivalence, read work, storage and write maintenance. Refresh planner statistics with ANALYZE or the appropriate PRAGMA optimize workflow and inspect the resulting plan again. Statistics help the planner estimate the choices available to it. Retain these measurements when the query, schema or data distribution changes, so that later adjustments can be compared with a clear baseline.

Slide 20 Choose the index that matches the work to be done

The final matrix connects query requirements with index properties. Selective equality benefits from a usable leading key. Ordered, limited results benefit when later key fields provide the requested order after earlier fields are fixed by equality. Covering access supplies required values directly. A partial index represents a subset described by a predicate that the query satisfies.

Read and write patterns determine how these properties combine into a useful design. The experiments showed that a small returned result can require very different work depending on search, ordering and table lookups. Keep that physical route in view. An index helps by narrowing, ordering or supplying data along the path that the query actually follows.